TAD Polinômios + Testes automatizados

Polinômios são expressões algébricas, desde aquelas que envolvem apenas números até as que apresentam letras, potências, coeficientes, entre outros elementos. Os polinômios são formados a partir da seguinte característica:

- O polinômio pode ter **n** termos, onde cada termo tem expoente ligado à variável **x**, e todos os expoentes devem ser **números naturais**.
- a_n, a_{n-1}, … a_1 e a_0 são números reais chamados coeficientes;
- a_0 é o termo independente de **x**.
- x é a variável.

O grau de um polinômio é o expoente máximo que ele possui. Se $a_n \neq 0$, então o expoente máximo **n** é dito grau do polinômio e indicamos **grau(P) = n**.

Exemplos:

- $P(x) = 5$ ou $P(x) = 5x^0$ é um polinômio constante, ou seja, $\text{grau}(P) = 0$.
- $P(x) = 3x + 5$ é um polinômio do 1º grau, isto é, $\text{grau}(P) = 1$.
- $P(x) = 4x^3+7x^2$ é um polinômio do 3º grau, ou seja, $\text{grau}(P) = 3$

Generalizando um polinômio P de grau(P)= n tal que **n**≥0 pode ser descrito como abaixo

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0 x^0$$

Por exemplo $P(x) = 4x^6 + 3x^5 - 6x^3 - 8x^2 + 3x + 4$

Objetivo do trabalho

O objetivo desta atividade é implementar o TAD (Tipo Abstrato de Dado) Polinômio, no **paradigma imperativo**, que é representação computacional de uma entidade matemática real:

Antes de começar a implementar verifique os arquivos abaixo:

`poli.h`: contém a definição do TAD Polinômio bem como as interfaces das operações.

`poli.c`: é nesse arquivo que você irá implementar as operações do seu TAD

`teste_poli.c`: que faz o teste das operações do TAD Polinômio.

Os testes descritos no arquivo `teste_poli.c` deverá lhe guiar e orientar na implementação do TAD, garantindo que suas operações atendam às especificações definidas. Para você conseguir implementar as operações, seguem abaixo uma breve explicação das operações matemáticas de um polinômio.

a) **Calcular o valor de polinômio:** basta substituir o valor de x na expressão e efetuar as operações.

   Exemplo: Se $P(x) = x^3 - 5x + 2$ para $x = -1$ temos $P(-1) = 6$.

b) **Calcular a soma de dois polinômios**: Para calcula a soma de dois polinômios basta somar os coeficientes dos termos que possuem o mesmo expoente. Considere os polinômios P e Q:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0 x^0$$
$$Q(x) = b_n x^n + b_{n-1} x^{n-1} + \ldots + b_1 x^1 + b_0 x^0$$

   A adição pode ser definida por:

$P(x) + Q(x) = (a_n + b_n)x^n\ (a_{n-1} + b_{n-1})x^{n-1} + \ldots + (a_1 + b_1)x^1 + (a_0 + b_0)\ x^0$

Veja o exemplo abaixo:

$$\begin{array}{l} 4x^6 + 3x^5 - 6x^3 - 8x^2 + 3x \\ -x^6 + 6x^4 + 2x^3 - 4x^2 + 9 \\ \hline 3x^6 + 3x^5 + 6x^4 - 4x^3 - 12x^2 + 3x + 9 \end{array}$$

c) **Calcular a multiplicação entre dois polinômios**: Nessa operação devemos empregar a propriedade distributiva multiplicando cada termo do primeiro polinômio por cada termo do segundo polinômio.

Considere os polinômios P e Q:

$P(x) = a_n x^n + a_{n-1}\ x^{n-1} + \ldots + a_1\ x^1 + a_0\ x^0$
$Q(x) = b_n x^n + b_{n-1}\ x^{n-1} + \ldots + b_1\ x^1 + b_0\ x^0$

A multiplicação pode ser definida por:

$$\begin{aligned} P(x) * Q(x) = {} & a_n x^n\ (b_n x^n + b_{n-1}\ x^{n-1} + \ldots + b_1\ x^1 + b_0\ x^0) + \\ & a_{n-1}\ x^{n-1}\ (b_n x^n + b_{n-1}\ x^{n-1} + \ldots + b_1\ x^1 + b_0\ x^0) + \\ & \ldots\ldots\ldots\ldots \\ & a_0\ x^0\ (b_n x^n + b_{n-1}\ x^{n-1} + \ldots + b_1\ x^1 + b_0\ x^0) \end{aligned}$$

Veja o exemplo abaixo:

$(2x^2+3x+3) * (x^2+4x+1) = 2x^2*(x^2+4x+1) + 3x*(x^2+4x+1) + 3*(x^2+4x+1)$
$=2x^4+8x^3+2x^2+3x^3+12x^2+3x+3x^2+12x+3$
$=2x^4+ 11x^3+ 17x^2+15x +3$

Importante: caso o polinômio resultante tenha termos com o mesmo expoente esses devem ser somados ou subtraídos para obter o resultado final.

d) **Calcular a divisão entre dois polinômios**: A divisão de polinômios é realizada utilizando o seguinte algoritmo: dividir o termo de maior grau do **dividendo** pelo do **divisor**, multiplica-se, subtrai-se e repete-se o processo até encontrar o **quociente**.

Considere os polinômios P e Q:

$P(x) = a_n x^n + a_{n-1}\ x^{n-1} + \ldots + a_1\ x^1 + a_0\ x^0$
$Q(x) = b_n x^n + b_{n-1}\ x^{n-1} + \ldots + b_1\ x^1 + b_0\ x^0$

Onde $P(x)$ é o **dividendo** e $Q(x)$ é o **divisor**.

Algoritmo para dividir dois polinômios

- **Dividir:** Divida o termo de maior grau de pelo termo de maior grau de para encontrar o primeiro termo.
- **Multiplicar e Subtrair:** Multiplique esse termo pelo **divisor** e subtraia do **dividendo** e encontre o primeiro resto parcial.
- **Repetir:** Repita o processo usando o resto parcial como novo dividendo até que o grau do resto seja inferior ao grau do divisor

Exemplo:
Dividir $P(x) = 2x^2 + 5x + 3$ (**dividendo**) por $Q(x) = x + 1$ (**divisor**)

1ª iteração
1.1 $2x^2 / x = 2x$ (**2x** é primeiro termo do **quociente**).
2.1 $2x * (x + 1) = 2x^2 + 2x$ (multiplique esse termo pelo **divisor**)
3.1 $(2x^2 + 5x + 3) - (2x^2 + 2x) = 3x + 3$ (subtraia do **dividendo** e encontre o primeiro **resto** parcial)
repetir até que o grau do **resto** seja inferior do grau do **divisor**.

2ª iteração
1.2 $3x / x = 3$ (3 é segundo termo do **quociente**).
2.2 $3 * (x + 1) = 3x + 3$ (multiplique esse termo pelo **divisor**)
2.3 $(3x + 3) - (3x + 3) = 0$ (subtraia do **dividendo** e encontre o primeiro **resto** parcial).
grau do **resto inferior** ao grau do **divisor**

resultado final: retorne o **quociente** $2x + 3$

Critérios de Avaliação do Trabalho:

- O trabalho deve ser desenvolvido na **linguagem C** e será testado usando o compilador GCC do instalado na versão Ubuntu-22.04, caso não tenha o Ubuntu instalado consulte o documento **`WSL_Windows_Ubuntu_VSCode.pdf`** disponível no Moodle.
- Você só pode modificar o arquivo **`poli.c`**, para compilar todo os arquivos fontes use o seguinte comando abaixo:

```
gcc -g poli.c teste_poli.c -o teste_poli
```

- Caso o programa não compile ou não execute **será atribuída a nota 0 ao trabalho.**
- Caso programa apresentarem ***warning*** durante a compilação ou não finalize com retorno igual a 0, será descontado 1.0 (um ponto) por ***warning*** relatado.
- O programa não pode conter erro de acesso memória, caso tenha a nota será reduzida à metade. Para testar a memória do seu programa use o **`valgrind`** conforme descrito abaixo:

```
valgrind --leak-check=yes ./teste_poli
```

- Você deve implementar as operações de forma que atenda qualquer caso de entrada conforme o enunciado do trabalho, além disso as implementações das operações devem passar nos testes proposto para que seja considera válida, qualquer tentativa de burlar os testes será atribuída nota zero ao trabalho
- Nos testes já é dada uma previsão de nota, caso seja necessário, para esclarecer alguma dúvida quanto a implementação da atividade, poderá ser **requisitado uma entrevista com os integrantes** do grupo, com intuito de validar se as regras acima foram todas atendidas.

Observações importantes.

- O programa deve ser devidamente documentado e poderá ser desenvolvido em grupo de **até dois alunos**. É **imprescindível** que os nomes dos integrantes do grupo sejam mencionados no início do arquivo fonte do trabalho. Além disso, deve-se seguir as **Orientações para Desenvolvimento de Trabalhos Práticos**, as quais estão disponíveis no Moodle.

- Fica **terminantemente proibido** o uso de ferramentas de **Inteligência Artificial**, como o ChatGPT, para a **geração automática do código do projeto**. Qualquer tentativa de burlar esta restrição será considerada uma infração disciplinar, conforme o **COMUNICADO DA FCI de 05/02/2025**.